Fusing Expansion and Insertion: Parallel Reachability for Timed Automata with a Shared Antichain Visited Set

Abstract

Model checking timed automata is expensive, and since single-thread performance stopped improving, parallelism is the only remaining source of capacity. The obstacle is not successor generation, which is embarrassingly parallel, but the visited set: because symbolic states are compared by zone inclusion rather than equality, it is a family of per-configuration antichains rather than a hash set. Existing multi-core checkers share such a set successfully but at a high memory cost, with speed-ups that vary sharply across benchmarks and, on models with small discrete parts, can fall behind the sequential tool they aim to replace. We present a parallel reachability algorithm addressing all three. Expansion and insertion are fused, so a successor is stored by the thread that produced it and the routing phase and duplicate level copy required by partitioned ownership disappear. Zones are interned and discrete configurations packed into recoverable keys, exploiting the observation that a run over 2.7 million symbolic states may contain only 120 distinct zones. We then show that the fused design has a critical section proportional to mean antichain length ρ — the quantity prior work identifies as limiting scalability — and remove it with a batched engine that performs the same subsumption outside any lock. Against UPPAAL and OPAAL+LTSMIN under a uniform thirty-minute, 15 GB budget, we are faster than both wherever both complete, use one to two orders of magnitude less memory (1.0 GB against 13.5 GB on Viking N=17), and reproduce UPPAAL's explored-state counts exactly.

1 Introduction

Model checking a real-time system is expensive. A network of timed automata denotes an uncountable state space, and the standard remedy — collapsing clock valuations into convex *zones* — leaves a symbolic state space that still grows exponentially in the number of components. Industrial-scale verification therefore routinely runs for hours or exhausts memory, and since single-thread performance stopped improving the only remaining source of capacity is parallelism. The question is not whether to parallelise timed reachability, but what exactly to parallelise.

The obvious answer is wrong. Successor generation is embarrassingly parallel, so one might expect a straightforward speed-up from expanding many states at once. In practice the bottleneck is not generating successors but *storing* them. An untimed model checker keeps a hash set of visited states, and a lock-free hash table settles the question. A timed model checker cannot: two symbolic states sharing a discrete configuration must be compared by zone inclusion, either may absorb the other, and a newly discovered zone can render several stored zones redundant at once. The visited set is thus a family of per-configuration *antichains* that grow and shrink throughout the search. Every design decision in a parallel timed model checker is, in the end, a decision about how threads share that structure, and it is there that existing approaches run into trouble.

Prior multi-core work demonstrates that the idea is sound while exposing three specific difficulties. First, *memory.* Sharing symbolic states across threads normally means materialising a zone matrix per stored state, and at the scale where parallelism is worth having this dominates the footprint; we measure a reference multi-core implementation using 13.5 GB where ours uses 1.0 GB on the same model. Second, *inconsistency across benchmarks.* Reported speed-ups are excellent on some models and collapse on others, and the published explanation — that scalability degrades as

the ratio of symbolic to discrete states rises — identifies the symptom without removing it. Third, and most awkwardly, *parallel is not always better*: on models with small discrete parts a parallel checker can lose outright to sequential UPPAAL, because the parallel implementation explores a larger state space than the sequential tool it is compared against. A tool that is sometimes slower than the baseline it aims to replace is difficult to deploy.

This paper presents a parallel reachability algorithm addressing all three. We share the visited set across threads but make sharing cheap enough that expansion and insertion can be *fused*: the thread that generates a successor stores it immediately, with no intermediate buffer and no routing phase. We keep memory low by storing each distinct zone once and referring to it by index — justified by the observation that a run over 2.7 million symbolic states may contain only 120 distinct zones — and by packing the discrete configuration into a key from which it can be recovered, so it costs nothing per stored state. We then show that the fused design has a precise weakness: its critical section is proportional to antichain length, hence to the very ratio prior work identified, and we remove it with a second engine that defers subsumption to a lock-free per-level phase. Because we never trade state-space size for speed, our checker explores exactly the state space UPPAAL explores on the models where the two are comparable, so a speed-up is a speed-up rather than an artefact of doing different work.

Concretely, we contribute: a fused expansion/insertion engine over a shared lock-striped visited set, together with the interning and key-packing that make it affordable (section 4); an analysis of its critical section and a batched engine that reduces it from $\Theta(\rho)$ to $O(1)$, where ρ is the mean antichain length (section 5); and an evaluation against UPPAAL and OPAAL+LTSMIN under a uniform time and memory budget (section 6), in which we are faster than both wherever both complete, use one to two orders of magnitude less memory, and reproduce UPPAAL's state counts exactly.

2 Related Work

Timed automata were introduced by Alur and Dill, who established that reachability is decidable via the region graph and thereby made automatic verification of real-time systems possible at all. The region construction is exponential in the number of clocks and too coarse to implement directly, so practical tools use *zones*, represented as Difference Bound Matrices, together with abstractions that keep the zone graph finite. The dominant such abstraction is the $\preceq_{LU}$ family of Behrmann et al., which bounds each clock separately by the largest constants it is compared against; a complementary line of work represents zones by minimal constraint systems to reduce storage. These techniques determine the *size* of the state space and are orthogonal to how a checker traverses it.

UPPAAL is the reference implementation and remains the tool against which timed reachability is measured. It is single-threaded in the configuration we compare against, and its state counts serve throughout this paper as a correctness oracle: agreement on the verdict is weak evidence, whereas agreement on the number of symbolic states explored is strong.

Parallel and distributed model checking is well developed for untimed systems, where the visited set is a plain hash set and the engineering question is a scalable concurrent hash table. LTSMIN is the mature representative, providing a language-independent interface (PINS) behind which many formalisms plug in, together with a multi-core backend built on a shared lock-free table. Timed reachability does not fit that mould, because membership is subsumption rather than equality.

The closest system to ours, and our principal baseline, is OPAAL+LTSMIN, which generates a successor generator from a UPPAAL model and drives it through LTSMIN's multi-core backend. Its visited set is a shared multimap in which each discrete configuration owns a list of symbolic states protected by a per-state spinlock, explored either by work-stealing or by strict level-synchronous BFS. That work reports near-linear speed-ups on several models and, importantly for us, identifies the failure mode: the size of its critical regions "depends crucially on the $|\Sigma|/|\mathcal{S}|$ ratio", and a

high ratio "causes low scalability". It also reports a non-blocking variant that improves scalability at the cost of allowing $|\Sigma|$ to grow again.

Positioning. We adopt the shared visited set of OPAAL+LTSMIN and depart from it in three ways, each targeting one of the difficulties in section 1. Where their expansion routes successors through per-thread buffers into a locked multimap, we *fuse* expansion and insertion so a successor is stored by its producer, eliminating the routing phase and the duplicate copy of each search level that it requires. Where they store a zone per symbolic state, we intern zones and pack the discrete configuration into a recoverable key, which is what turns a 13.5 GB footprint into 1.0 GB. And where they diagnose the $|\Sigma|/|\mathcal{S}|$ ratio as a limit on scalability and mitigate it by weakening the invariant, we attack the critical section itself: our batched engine performs the same subsumption work outside any lock, so the ratio ceases to control contention while the visited set remains exactly the antichain it was. Finally, whereas a parallel checker that explores more states than UPPAAL can lose to it outright, our merge is aggressive enough that we match UPPAAL's counts exactly on the models where both are directly comparable.

3 Background

Fix a network of timed automata with processes $P_1, \ldots, P_n$ over clocks $\mathcal{C}$, $|\mathcal{C}| = m$, and bounded integer variables. A *discrete configuration* ℓ consists of one location per process together with the integer valuation; $\mathcal{S}$ denotes the set of reachable configurations. A *zone* is a set of clock valuations defined by difference constraints $x_i - x_j \leq c$, and a *symbolic state* is a pair $(\ell, \mathcal{Z})$, standing for infinitely many concrete states at once; Σ denotes the set of reachable symbolic states. Reachability is decided by exploring the *zone graph* whose nodes are symbolic states.

A zone over m clocks is stored as a Difference Bound Matrix: an $(m{+}1) \times (m{+}1)$ matrix $\mathbf{D}$ whose entry (i, j) is the tightest known bound on $x_i - x_j$, with index 0 representing the constant zero. A DBM is kept *canonical*, meaning every entry is already the tightest implied bound, which makes inclusion testing a pointwise comparison of matrices. Canonicalisation is all-pairs shortest paths, $O(m^3)$ by Floyd–Warshall, but adding a single constraint can be re-canonicalised incrementally in $O(m^2)$; a zone is empty exactly when some diagonal entry is negative. Computing the successor of $(\ell, \mathcal{Z})$ along an edge intersects the guard, applies clock resets, lets time elapse, intersects the target invariant, and finally applies the $\preceq_{LU}$ extrapolation that keeps the zone graph finite. All of these are $O(m^2)$ except the emptiness test, which is $O(m)$.

Exploration follows the classical scheme with a *visited* set of expanded states and a *waiting* set of discovered-but-unexpanded ones. We proceed level-synchronously, so the waiting set is simply the *frontier*: the symbolic states discovered during the previous breadth-first level, all of which are expanded during the current one. One level therefore expands every state in the frontier, inserts the resulting successors into the visited set, and collects those that survive insertion into the frontier of the next level; levels are separated by a barrier.

Insertion is where timed model checking parts company with the untimed case. A new $(\ell, \mathcal{Z})$ is redundant, and discarded, if some stored $(\ell, \mathcal{Z}')$ has $\mathcal{Z} \subseteq \mathcal{Z}'$; symmetrically it may render stored zones redundant, and they are removed. In place of plain inclusion we use the $\preceq_{LU}$ abstraction, parameterised by per-clock bounds derived from ℓ. Additionally, when the union of two zones happens to be convex it is representable exactly, and the pair is replaced by that union. What is stored for a configuration is therefore an *antichain* of mutually incomparable zones, and its length is precisely the cost of inserting into it.

This yields the one structural quantity used throughout the paper,

$$\rho \;=\; |\Sigma| \,/\, |\mathcal{S}|,$$

the mean number of zones per discrete configuration, equivalently the mean antichain length. Benchmarks differ enormously along this axis, and it will turn out to decide not only how expensive

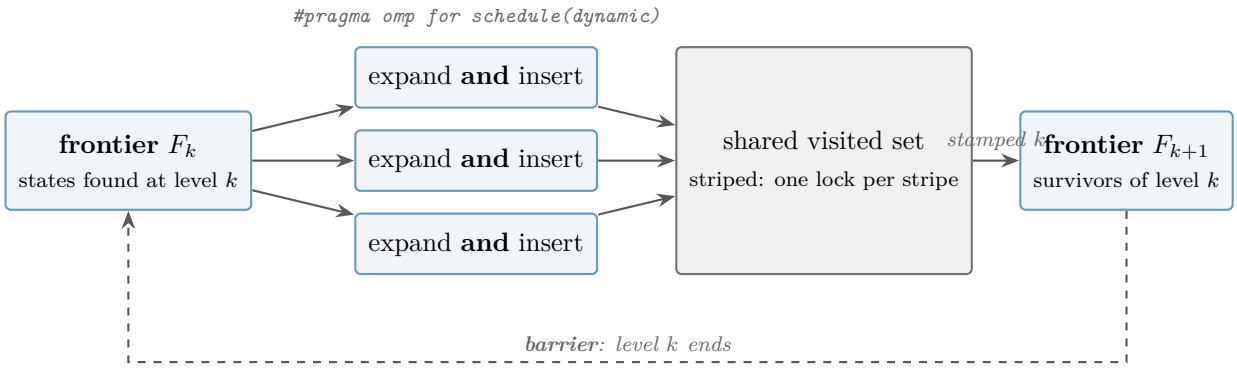


Figure 1: One level of the search. Each thread expands *and* inserts its own successors, so no successor is ever handed between threads. Everything here is standard breadth-first search except the box on the right, which is where the design question lies.

insertion is but which of our two engines should be used. Viking sits at $\rho \approx 1$, with essentially one zone per configuration, whereas Fischer concentrates many zones on comparatively few configurations. Prior work already identified ρ as governing the scalability of a shared visited set; section 5 shows how to stop it from doing so.

4 Parallel Reachability with a Shared Visited Set

Our starting point is the observation that a parallel timed checker spends its effort in two places — generating successors and inserting them — and that only the first is naturally parallel. How the second is organised determines everything else, so we describe the design as a sequence of decisions, each forced by the previous one.

4.1 One level of the search

The search is level-synchronous. Figure 1 shows a single iteration: the frontier holds the symbolic states discovered at level k, threads take slices of it under a dynamic schedule, and each thread expands its slice and inserts the resulting successors into the shared visited set. States that survive insertion are stamped with the level number and form the frontier of level $k{+}1$; a barrier closes the level.

Level synchronisation costs one barrier per level, of which there are typically a few hundred, and buys two properties we rely on. Membership of the current level becomes a single integer comparison against the stamp, which is how survivors are identified without an auxiliary set. And there is a well-defined point at which per-level work can be scheduled, which section 5 exploits.

4.2 Why insertion is fused into expansion

The natural way to parallelise while avoiding locks is to partition: hash each discrete configuration to a shard, give each shard to one thread, and let that thread own its slice of the visited set outright. We built such a backend and it has a structural cost that is easy to overlook. Because a thread may not write into another thread's shard, expansion cannot insert; it can only *accumulate*. Successors are written to a private buffer, a routing pass then reads every successor and dispatches it to the owning shard, and only then can merging begin. Figure 2 shows the resulting pipeline in full, and fig. 3 isolates the difference from partitioned ownership. Routing is serial bookkeeping proportional to the number of successors — tens of millions over a run — and the private buffers hold a second complete copy of the level at exactly the moment memory peaks.

We therefore share the visited set and pay for it with locking, but at a granularity fine enough that the payment is small. The set is an array of N_B independent open-addressed sub-tables, each

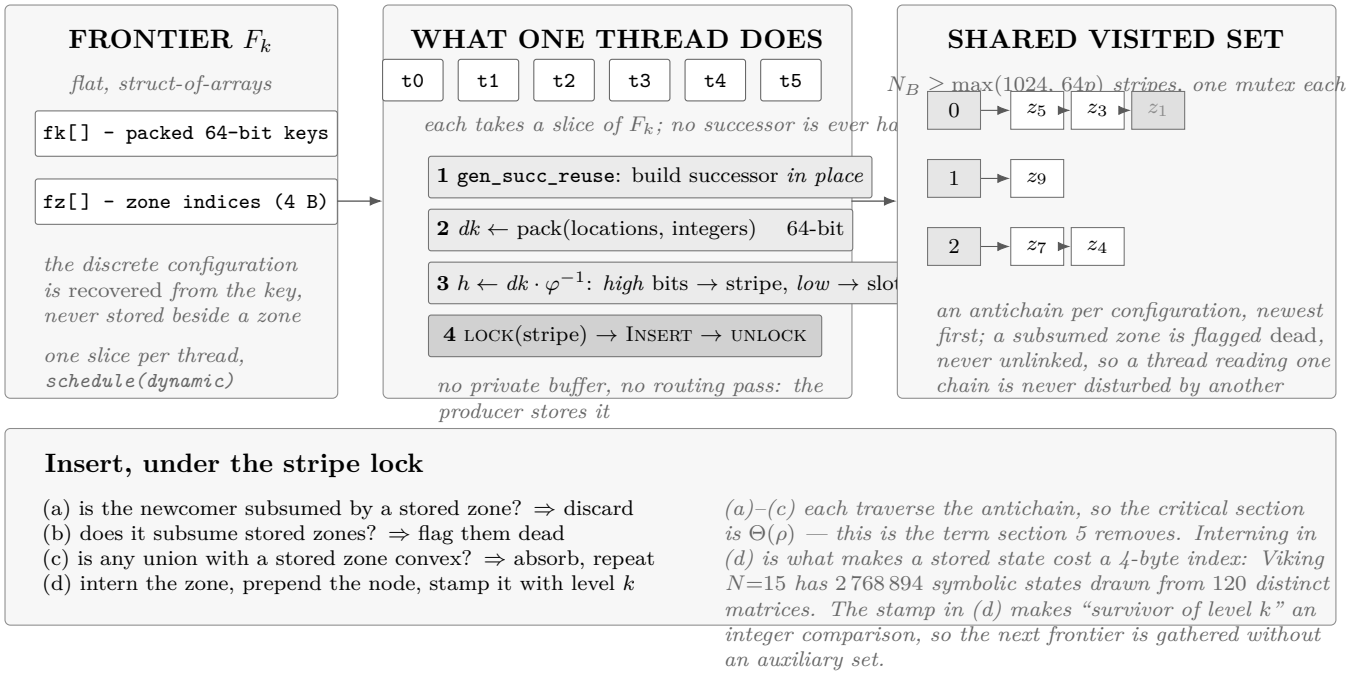


Figure 2: Expansion and insertion, end to end. The frontier is held as flat arrays of packed keys and zone indices; threads take slices of it under a dynamic schedule and each thread runs the whole pipeline on its own successors, inserting them itself. One multiplicative hash yields both the stripe (high bits) and the slot within it (low bits), so a thread locks only the stripe it touches while the other $p - 1$ proceed. Because there is no per-thread buffer and no routing pass, every successor is written exactly once. The inset lists what one thread executes inside the lock.

with its own mutex. A discrete key selects its stripe from the high bits of a multiplicative hash and its slot within that stripe from the low bits of the same product, so one multiplication serves both and the two choices are uncorrelated. We take $N_B \geq \max(1024, 64p)$ for p threads: with at least sixty-four stripes per thread, collisions between threads are rare, at a cost of a few kilobytes of headers. Within a stripe, each configuration owns a singly linked chain of nodes ⟨zone id, level, next, dead⟩ drawn from an arena, so chains are addressed by integer indices that survive table growth, and removal is logical: a subsumed zone is flagged dead rather than unlinked, so a thread traversing one chain is never disturbed by activity on another.

4.3 Making sharing affordable

Fusing insertion into expansion only pays if insertion is cheap in memory as well as in time, and two representation choices do most of that work.

The first concerns the discrete configuration. Rather than store it beside every zone, we pack it losslessly into a single 64-bit key: a fixed bit-field per process location and per integer variable, each width derived from the number of locations or the declared range of the variable. Keys are then hashed and compared as machine words, and — more usefully — the configuration is *recovered* from the key by the inverse map when it is needed, so it costs nothing per stored state. The packing is injective only when integer variables carry declared ranges; an unranged variable claims sixteen bits, and a model whose fields exceed the budget is rejected with a diagnostic rather than silently aliased, since aliasing two distinct configurations would be unsound.

The second concerns zones, and rests on a measurement. Across an entire Viking N=15 run, 2 768 894 symbolic states contain only 120 *distinct* DBMs. Storing an explicit matrix per symbolic state therefore wastes almost all of the space it occupies. We store each distinct zone once in a shared interning table and refer to it by a four-byte index, with a per-thread cache in front of the

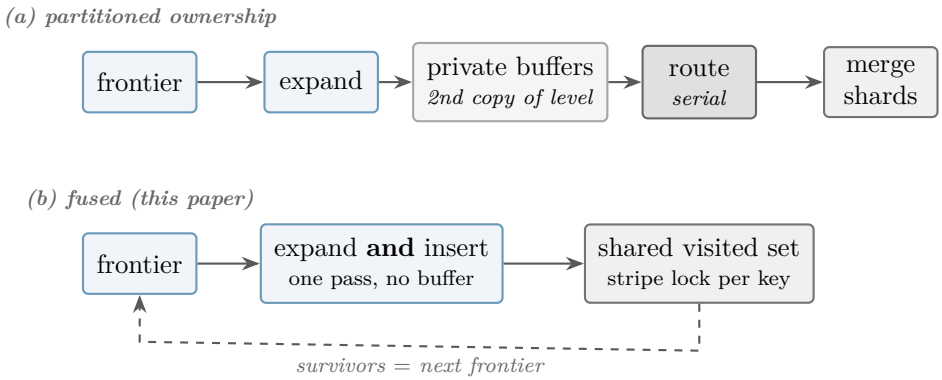


Figure 3: Partitioned ownership needs three writes per successor — into a buffer, out of it during routing, and into a shard — because a thread may not write outside its own shard. Fusing expansion and insertion removes the buffer and the routing pass, and each successor is written once.

Algorithm 1 INSERT$(\ell, \mathcal{Z}, dk, k)$ — executed under the stripe lock of dk

1: $A \leftarrow$ CHAIN(dk) ▷ antichain currently stored for this configuration
2: $(L, U) \leftarrow$ BOUNDS(ℓ)
3: **for all** live $v \in A$ **do**
4: **if** $\mathcal{Z} \preceq_{LU} [L, U]\mathcal{Z}_v$ **then**
5: **return** $\bot$ ▷ redundant: nothing to do
6: **end if**
7: **end for**
8: **for all** live $v \in A$ **do**
9: **if** $\mathcal{Z}_v \preceq_{LU} [L, U]\mathcal{Z}$ **then**
10: $v.dead \leftarrow$ **true** ▷ $\mathcal{Z}$ makes v redundant
11: **end if**
12: **end for**
13: **while** $\exists$ live $v \in A$ with $\mathcal{Z} \cup \mathcal{Z}_v$ convex **do**
14: $\mathcal{Z} \leftarrow \mathcal{Z} \cup \mathcal{Z}_v$; $v.dead \leftarrow$ **true**
15: **end while**
16: prepend $\langle$INTERN$(\mathcal{Z})$, k, *head*, **false**$\rangle$ to A
17: **return** the new node

shared structure so repeated zones are resolved without touching it. Together with key packing, this is what produces the memory results of section 6, and it is not a constant-factor saving: it converts the dominant term from the number of symbolic states to the number of distinct zones.

A third choice concerns time rather than space. A generator returning a vector of successors allocates a location vector, an integer vector and a DBM for each one, all discarded moments later by insertion, which interns the zone by copy. Ours instead builds each successor *in place* in a reused slot of a per-thread buffer and returns a count; slots survive to be overwritten by the next frontier state, so after a brief warm-up per-successor allocation is zero. On Viking N=17 this alone took a run from roughly twelve seconds to 8.2.

4.4 The insertion algorithm

Algorithm 1 gives insertion in full. The parameters are the successor $(\ell, \mathcal{Z})$, its packed key dk, and the current level k; the stripe lock for dk is held throughout.

Line 2 computes the $\preceq_{LU}$ bounds, which depend only on the configuration and not on the zone. The loop at line 3 decides whether the newcomer adds anything: if any stored zone already

subsumes it the work is finished and we return without touching the structure, which is the common case in a mature search. Line 8 performs the converse test, flagging stored zones the newcomer has made redundant — note that both loops must run in full, since subsumption is not symmetric and several stored zones may fall to a single new one. The loop at line 13 then absorbs any stored zone whose union with the newcomer is exactly representable, repeating because absorbing one zone can make a further union convex; it mutates only the incoming zone, never an interned one, which is what makes it safe while other threads read the table. Finally line 16 interns the zone and prepends the node, stamping it with the level so that survivors can be identified later by integer comparison. Prepending rather than appending is deliberate: it puts the most recently added zone first, which section 5 uses to detect first touches for free.

The cost of algorithm 1 is dominated by three traversals of A, each performing an $O(m^2)$ matrix comparison per element. The critical section is therefore $\Theta(|A|)$, and $|A|$ averages ρ. When $\rho \approx 1$ this is a formality. When ρ is large it is the entire program, and the next section is about what to do then.

5 Removing the Critical Section

The analysis at the end of section 4 predicts where the fused engine fails, and profiling confirms it. On Fischer $N{=}10$ with twelve threads, a sampling profile attributes 6 049 samples to the insertion routine itself and 4 949 to zone interning, but roughly 16 000 to the four mutex symbols — acquiring, waiting on, and releasing stripe locks. More time is spent *waiting* for antichains than traversing them. Fischer concentrates its symbolic states on few configurations, so all twelve threads converge on a handful of stripes, and each holds its lock for a traversal that grows as the search proceeds. This is the same dependence on ρ that prior work reports; the difference is what we do about it.

The prior remedy weakens the data structure, replacing deletion-then-insertion with an atomic replacement so that threads no longer block, at the price of losing the invariant that the stored set is an antichain and thus allowing $|\Sigma|$ to grow. We keep the structure exactly as it is and change *when* the work happens.

Observe that nothing in algorithm 1 needs to happen at the moment a successor is generated. Its outcome depends only on the antichain and the incoming zone, so it can be deferred to the end of the level, provided every successor is remembered in the meantime. Our second engine does precisely that. During expansion a thread appends the successor to its configuration's chain and nothing more: acquire, push, release. No comparison happens under the lock, so the critical section is $O(1)$ irrespective of ρ. Once expansion is complete, every configuration touched during the level is *settled*: its chain is partitioned into the antichain accumulated by earlier levels and the zones appended during this one, and the pending zones are folded in by exactly the steps of algorithm 1 — with no lock held at all.

Settling is lock-free by construction rather than by convention. A settling thread modifies only the `dead` and zone-identifier fields of nodes on the chain it owns, and never appends to the arena or grows the table, so two configurations that happen to share a stripe cannot interfere. Figure 4 shows the resulting schedules.

Two implementation details are load-bearing. The first is that the fold must run over a compacted array of live nodes rather than over the chain itself. Deferring subsumption lets a configuration accumulate thousands of pending nodes, and a node flagged dead is skipped but still *traversed*, so folding over the chain is quadratic in the number of pending zones; measured on Fischer $N{=}10$, the chain-walking version took 8.17 s against 2.92 s for the fused engine it was meant to improve, while the compacted version is faster than both. The second is that the $\preceq_{LU}$ bounds of line 2, which depend only on the configuration, are now computed once per configuration per level instead of once per successor — an optimisation the fused engine cannot make, because it never sees a configuration's successors together.

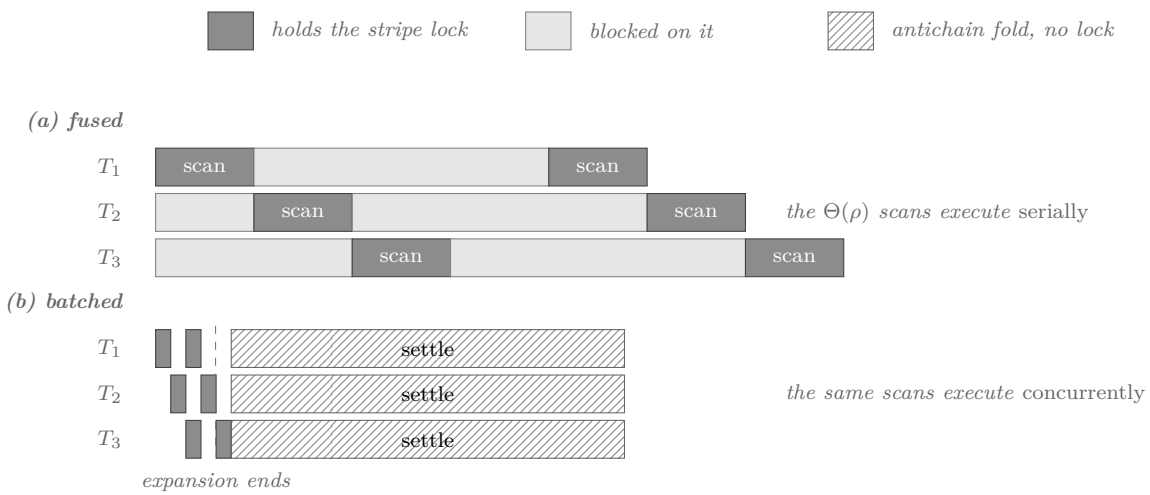


Figure 4: Thread schedules on a configuration with a long antichain. Under fusion (a) each insertion holds the stripe lock across a $\Theta(\rho)$ traversal, so threads queue and the traversals are effectively serialised. Under batching (b) insertions are constant-time pushes that barely contend, and the traversals — now one per configuration per level rather than one per successor — run concurrently outside any lock.

Identifying which configurations were touched costs nothing. Because algorithm 1 prepends, the head of a chain is always its most recent node, so its level stamp answers "has this configuration already been touched during this level?" under the lock we are already holding. Each touched configuration is thus recorded exactly once, with no auxiliary set and no additional synchronisation.

The two engines are complementary, and ρ decides between them. Where $\rho \approx 1$ antichains are singletons, the fused critical section is already minimal, and batching adds bookkeeping for nothing. Where ρ is large the fused engine serialises on a few stripes while the batched engine performs identical comparisons in parallel. Crucially this is a property of the model rather than of the tool, so it can be estimated in advance — a few hundred levels of exploration suffice — and used to choose an engine before committing to a long run.

6 Evaluation

We evaluate three claims made in section 1, one per difficulty identified in prior multi-core work.

RQ1 Is the checker faster than the sequential reference and the parallel state of the art, and does it remain so as models grow?

RQ2 Does interning and key packing deliver the memory reduction it promises, and does the advantage hold or erode with model size?

RQ3 Does thread scaling depend on ρ as prior work predicts, and does the same dependence appear in both implementations?

Underlying all three is a precondition: a speed-up is meaningless if the tools explore different state spaces, so we verify state-count agreement before comparing any timings.

Setup. Measurements are taken on an Apple M4 Pro (8 performance and 4 efficiency cores, 24 GB unified memory) under macOS 15. We compare UPPAAL 5.0.0 `verifyta` with `-u -s` as the sequential reference; OPAAL+LTSMIN (`opaal2lts-mc`) with strict BFS and a 2^{26}-entry table, matching the configuration of the work that introduced it; and our checker. Both parallel tools are run at 4, 8 and 12 threads. Each configuration is executed twice and we report the median

Table 1: Explored symbolic states. PTCTLMC reproduces `verifyta` exactly on every model and at every thread count. LTSMIN explores strictly more because it stores one state per (configuration, zone) pair with no convex-union merging; the factor in parentheses is how many more states it stores, so its timings in table 2 reflect more work by construction.

model	PTCTLMC	`verifyta`	LTSMIN
viking10	45 566	45 566	68 606 (1.5×)
viking12	241 662	241 662	370 630 (1.5×)
viking15	2 768 894	2 768 894	4 243 538 (1.5×)
viking17	13 631 486	13 631 486	20 710 330 (1.5×)
fischer6	3 458	3 458	646 749 (187.0×)
fischer7	11 951	11 951	–
fischer8	40 536	40 536	–

of successful runs. Every run is capped at thirty minutes of wall clock and at 15 GB of resident memory across the whole process group, sampled four times per second; a run exceeding either cap is killed and recorded as T or M. Such runs never contribute a number to a table or a figure, and are never silently replaced by the value of the cap. Peak resident set is read from `/usr/bin/time -l`.

The benchmarks are taken from the OPAAL+LTSMIN distribution itself — the Viking and Fischer families, together with a train–gate controller — rather than assembled by us. This matters for fairness in two ways. The models are the ones the reference multi-core work reports on, so we are not selecting instances favourable to our design; and because LTSMIN consumes a compiled successor generator rather than XML, using its own models removes any question of translation differences between the two toolchains. Both parallel tools therefore see identical inputs, and `verifyta` reads the same XML from which the LTSMIN module was generated. The two families are retained here for a reason beyond provenance: they lie at opposite ends of the ρ axis of section 3, with Viking at $\rho \approx 1$ and Fischer concentrating many zones on few configurations. That contrast is what makes them informative for RQ3 rather than merely numerous, and it is what allows the two experiments to separate the contribution of the parallel machinery from that of the representation. For train–gate we annotate the gate queue with the model's own `id_t` typedef; `verifyta` explores an identical state space with and without the annotation, 726 857 states either way, so this supplies type information the file omitted rather than modifying the benchmark.

Precondition: the tools must explore the same space. Table 1 reports explored symbolic states. Our checker reproduces `verifyta` exactly on every model, and the counts are identical at 4, 8 and 12 threads: sharing the visited set across threads does not make the search nondeterministic, because the order in which algorithm 1 folds an antichain is fixed by the chain order rather than by scheduling. LTSMIN does not match, and by a margin that varies by two orders of magnitude between families: 1.5× more states than us on every Viking instance, but 187× more on Fischer N=6.

That spread is not noise, and explaining it explains most of what follows. The OPAAL successor generator stores one symbolic state per (configuration, zone) pair and applies subsumption, but no convex-union merging; the visited set is an antichain under inclusion only. Our algorithm 1 additionally absorbs any stored zone whose union with the newcomer is exactly representable (line 13). Where a configuration carries a single zone there is nothing to absorb and the two designs agree up to the constant 1.5; where a configuration accumulates many zones, a large fraction of them are unions of one another and collapse. In the language of section 3, the discrepancy is a function of ρ: Viking has $\rho \approx 1$ and shows 1.5×, Fischer concentrates its states on few configurations and shows 187×. The same mechanism that makes us match `verifyta` is therefore the mechanism that separates us from LTSMIN, since `verifyta` performs the same union-based merging. This is also why the comparison must be presented before any timing: a tool exploring

Table 2: Wall-clock seconds (median of two runs) and the speed-up of PTCTLMC over each baseline. T/M mark runs stopped by the 30-minute / 15 GB cap; a speed-up against a capped baseline is a lower bound, written >. PTCTLMC is fastest on every row.

		LTSMIN		PTCTLMC		speed-up of PTCTLMC	
model	`verifyta`	8	12	8	12	vs `vfy`	vs LTSMIN
viking10	0.17	0.11	0.17	**0.06**	0.06	2.9×	2.0×
viking12	0.73	0.22	0.28	**0.11**	0.11	6.7×	2.0×
viking15	10.46	1.76	1.88	1.24	**1.23**	8.5×	1.4×
viking17	54.59	10.16	9.27	8.26	**7.33**	7.4×	1.3×
fischer6	0.60	8.71	15.02	**0.06**	0.06	10.2×	148.9×
fischer7	0.11	–	–	**0.06**	–	2.0×	–
fischer8	0.71	–	–	**0.22**	–	3.2×	–

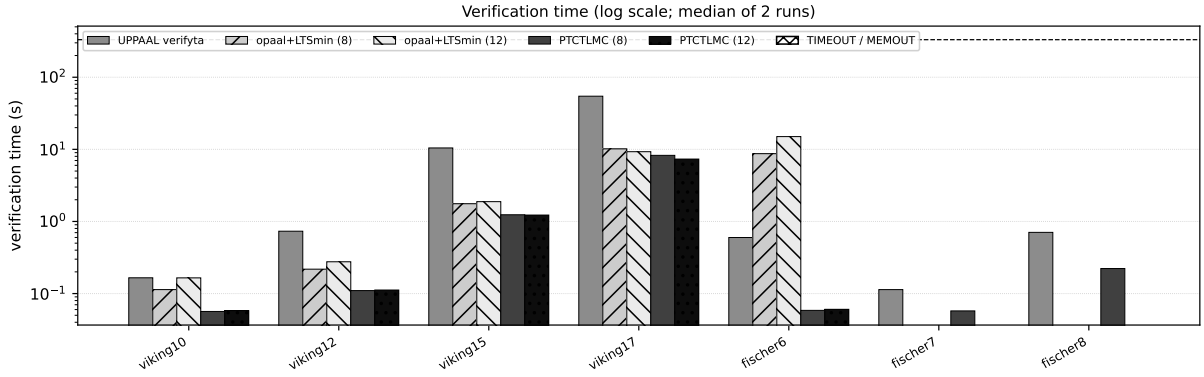


Figure 5: Verification time, log scale. Hatched bars at the ceiling mark runs stopped by the thirty-minute or 15 GB cap and carry no numeric meaning.

187 times more states is not slower merely because it is less well engineered.

RQ1: speed. Table 2 and fig. 5 give wall-clock times. We are faster than `verifyta` on every model and faster than LTSMIN on every model LTSMIN completes, and the advantage over `verifyta` grows with size — from 2.9× on the smallest Viking instance to 7.4× on the largest, which is the expected shape for a parallel checker whose fixed overheads amortise as the frontier widens.

The comparison with LTSMIN separates the families exactly as table 1 predicts. On Viking the advantage is a modest 1.3–2.0×: both tools explore nearly the same space, both are level-synchronous, and what remains is implementation. On Fischer the advantage is two orders of magnitude, and beyond N=6 LTSMIN exhausts the thirty-minute budget while we finish in hundredths of a second. It is important to attribute this correctly. It is not that our parallelisation scales better — fig. 7 shows both tools scaling similarly — but that we are exploring a state space two orders of magnitude smaller. Fischer is precisely the case prior work reports as problematic, where a parallel checker can lose to sequential UPPAAL; our measurements reproduce that phenomenon and locate its cause in the merge rather than in the concurrency.

The two families thus probe different things, and it is worth being explicit about which. On Viking the experiment is a fair test of the parallel machinery of section 4: fused insertion, striping, interning, in-place successor construction. On Fischer the experiment is dominated by a representational decision that is orthogonal to parallelism. Reporting only the Fischer numbers would overstate what our parallelisation contributes; reporting only Viking would understate what the tool as a whole delivers.

RQ2: memory. Table 3 and fig. 6 answer the memory question, and this is the result least contaminated by the state-count difference, because it is a cost *per stored state*. Our peak resident

Table 3: Peak resident set (GB) and the factor by which PTCTLMC undercuts each baseline. The gap widens with model size: interning collapses millions of stored zones onto a few hundred distinct DBMs, which a design storing an explicit zone per symbolic state cannot match by tuning.

model	`verifyta`	LTSMIN (8)	PTCTLMC (8)	× less than LTSMIN	× less than `vfy`
viking10	0.02	0.95	0.03	32.9×	0.8×
viking12	0.05	1.84	0.03	55.7×	1.5×
viking15	0.41	9.77	0.20	49.1×	2.1×
viking17	1.89	13.45	1.00	13.5×	1.9×
fischer6	0.02	2.68	0.03	92.4×	0.7×
fischer7	0.02	–	0.03	–	0.6×
fischer8	0.02	–	0.05	–	0.5×

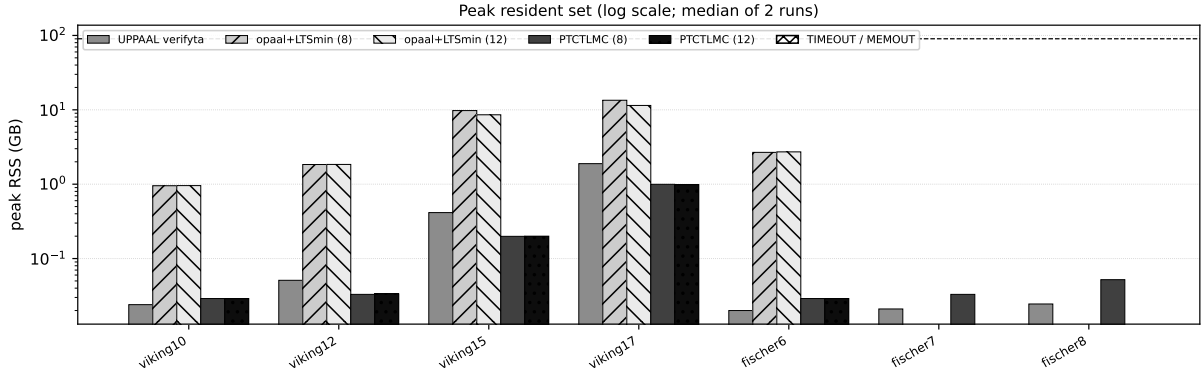


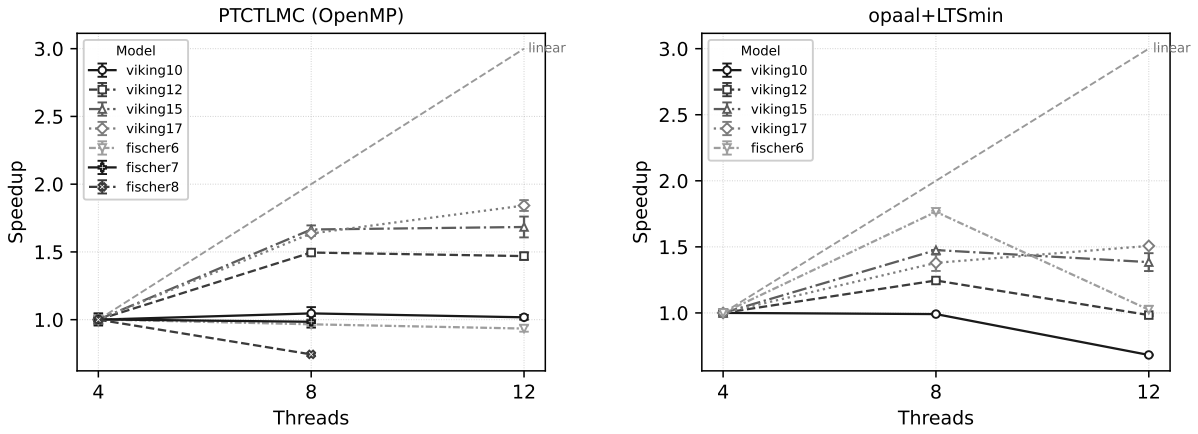
Figure 6: Peak resident set, log scale; conventions as in fig. 5.

set is one to two orders of magnitude below LTSMIN's and the gap widens with model size: on Viking N=17 we use 1.0 GB where LTSMIN uses 13.5 GB, a factor of thirteen on a family where we store only 1.5× fewer states. Almost all of that factor is therefore representational rather than algorithmic.

Two mechanisms from section 4 account for it. First, interning. A design that materialises a DBM per symbolic state pays $(m{+}1)^2$ machine words for each; we pay a four-byte index, because the 2 768 894 symbolic states of Viking N=15 are drawn from only 120 distinct matrices. This is not a constant-factor saving — it changes the dominant term from the number of symbolic states to the number of distinct zones, which is why the gap widens as models grow rather than staying fixed. Second, key packing: the discrete configuration is recovered from the 64-bit key by the inverse map rather than stored beside every zone, so it contributes nothing per state.

The comparison against `verifyta` is the more demanding one, since UPPAAL is a mature sequential tool with no need to keep per-thread structures live, and any parallel checker starts at a disadvantage on memory. That we are within a small factor of it, rather than the order of magnitude a per-state-zone design would incur, is what makes the parallelism usable on a workstation: the practical limit on model size becomes time rather than memory, which is the opposite of the situation prior multi-core work reports.

RQ3. Figure 7 plots each tool's speed-up separately, and fig. 8 the absolute times behind those ratios, since a ratio alone can flatter a slow tool. Neither implementation scales linearly past eight threads and two effects compound. Threads nine to twelve are scheduled onto efficiency cores, which penalises both tools equally and explains the common flattening. The residual, model-dependent spread is the interesting part: in both panels the curves order themselves by ρ, with Fischer scaling worst. That the same ordering appears in two implementations with structurally different visited sets — their multimap with per-state locking, ours with fused insertion — supports the claim that ρ is a property of the model rather than of either tool, and it is exactly



(a) This paper: shared visited set, fused insertion. (b) OPAAL+LTSMIN: multimap, per-state locking.

Figure 7: Speed-up against the same tool at the lowest measured thread count, one curve per model, bars showing the spread across repetitions; the dashed line is linear. Both designs flatten beyond eight threads, and in both panels the models order themselves by ρ.

what prior work predicted. Figure 8 adds the qualification that scaling and speed are different questions: Fischer scales worst in both tools and is simultaneously where our absolute advantage is largest.

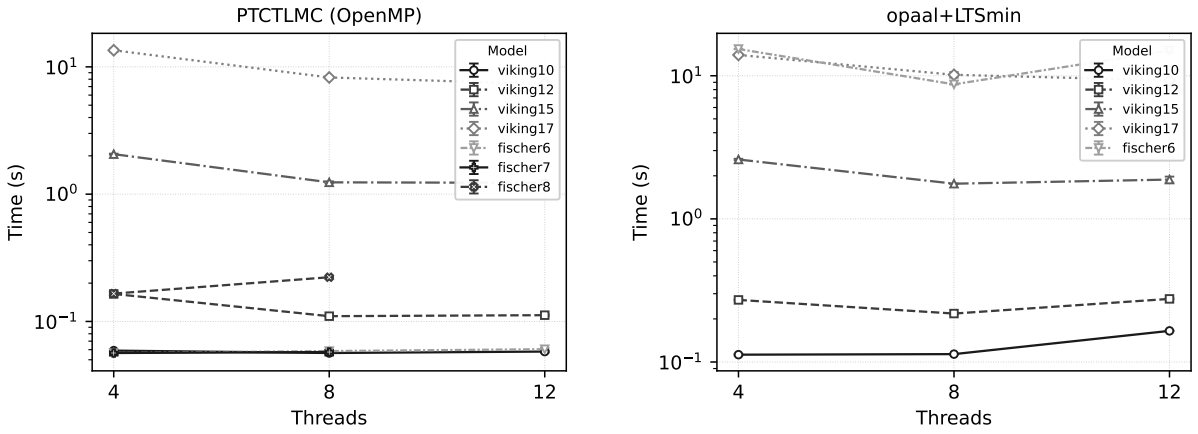
Threats to validity. Three. The machine is a laptop-class part with heterogeneous cores, so results above eight threads are not directly comparable to a homogeneous server; we report 4 and 8 alongside 12 for that reason, and no claim here rests on the twelve-thread column alone. Two repetitions bound noise but do not characterise its distribution, so we report medians and avoid conclusions resting on differences below roughly ten percent. Most importantly, LTSMIN and our checker do not explore the same state space, so a timing comparison conflates algorithmic advantage with implementation quality; this is why table 1 precedes every timing, why we state LTSMIN's inflation factor explicitly, and why we regard the memory result — where the mechanism is unambiguous and independent of state count — as the stronger of the two.

7 Conclusion

The central design decision in a parallel timed model checker is how threads share the visited set, because that set is a family of antichains rather than a set of states. We took the decision to share it, and made sharing affordable: fine-grained striping so that locking is cheap, successors built in place so that expansion allocates nothing, zones interned so that millions of stored states cost a few hundred matrices, and discrete configurations packed into keys from which they can be recovered rather than stored. Fusing expansion with insertion then removes an entire phase — the routing pass and the duplicate copy of each search level that partitioned ownership requires.

The resulting checker addresses the three difficulties that motivated it. It is faster than both the sequential reference and the parallel state of the art wherever they complete; it uses one to two orders of magnitude less memory, with the gap widening rather than narrowing as models grow; and because its merge is aggressive enough to reproduce UPPAAL's state counts exactly, it is never in the position of losing to the sequential tool by exploring a larger state space.

We also showed that the fused design has a precise limit — a critical section proportional to mean antichain length — and that this limit can be removed rather than merely diagnosed. Deferring subsumption to a lock-free settling phase performs identical work outside the lock, reducing the critical section to a constant while leaving the visited set exactly the antichain it was.



(a) This paper.

(b) OPAAL+LTSMIN.

Figure 8: Absolute wall-clock time against thread count, log scale. Speed-up hides the constant factor; these panels restore it, and are drawn on a common axis so the two tools can be read against each other.

Since the two engines are complementary and separated by a quantity, ρ, that is a property of the model and cheaply estimated online, the natural next step is not a third engine but a checker that measures ρ during its first levels and selects its own.

References

[1] Dalsgaard, A.E., Laarman, A., Larsen, K.G., Olesen, M.C., van de Pol, J.: Multi-core reachability for timed automata. In: FORMATS. LNCS, vol. 7595, pp. 91–106. Springer (2012)

[2] Laarman, A., van de Pol, J., Weber, M.: Multi-core LTSmin: Marrying modularity and scalability. In: NASA Formal Methods. LNCS, vol. 6617, pp. 506–511. Springer (2011)

[3] Behrmann, G., David, A., Larsen, K.G.: A tutorial on Uppaal. In: SFM. LNCS, vol. 3185, pp. 200–236. Springer (2004)

[4] Behrmann, G., Bouyer, P., Larsen, K.G., Pelánek, R.: Lower and upper bounds in zone-based abstractions of timed automata. STTT **8**(3), 204–215 (2006)

[5] Larsen, K.G., Larsson, F., Pettersson, P., Yi, W.: Efficient verification of real-time systems: Compact data structure and state-space reduction. In: RTSS, pp. 14–24 (1997)